جامعة السلطان مولاي سليمان
Université Sultan Moulay Slimane

المدرسة الوطنية للعلوم التطبيقية - بني ملال
Ecole Nationale des Sciences Appliquées - Béni Mellal

ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES

De Béni Mellal

TP 4

Architecture Orientée Services (SOA)

Distribution des Services

Réalisé par :

DIALLO THIERNO MOHAMMAD LAMINE

Encadrant :

Pr. BE ELBAGHAZAOUI

Année universitaire : 2025 — 2026

Introduction et Objectifs

Ce TP porte sur l'architecture orientée services (SOA), un modèle de conception qui consiste à décomposer une application en plusieurs services indépendants et communicants. Dans ce contexte, on développe une application e-commerce simple basée sur des microservices avec Spring Boot. Chaque service possède sa propre responsabilité et communique avec les autres via des API REST, tout en étant enregistré et découvert grâce à un serveur Eureka. Cette approche permet de simuler une architecture moderne, modulaire et évolutive, proche des systèmes utilisés dans les environnements professionnels.

Objectifs du TP

- Comprendre les principes fondamentaux de l'architecture SOA
- Mettre en place et manipuler des microservices avec Spring Boot
- Implémenter la communication entre services via REST et OpenFeign
- Utiliser Spring Data JPA pour gérer les données
- Mettre en place un service de découverte avec Eureka
- Tester et valider l'interaction entre les différents services à l'aide de Postman .

Architecture du système

Notre application e-commerce sera composée de quatre microservices :

Service	Description
Product Service	Gestion du catalogue de produits
Order Service	Gestion des commandes clients
Eureka Server	Service de découverte et registre des microservices
API Gateway	Point d'entrée unique pour les requêtes clients

Chaque service sera indépendant avec sa propre base de données (H2 en mémoire pour simplifier).

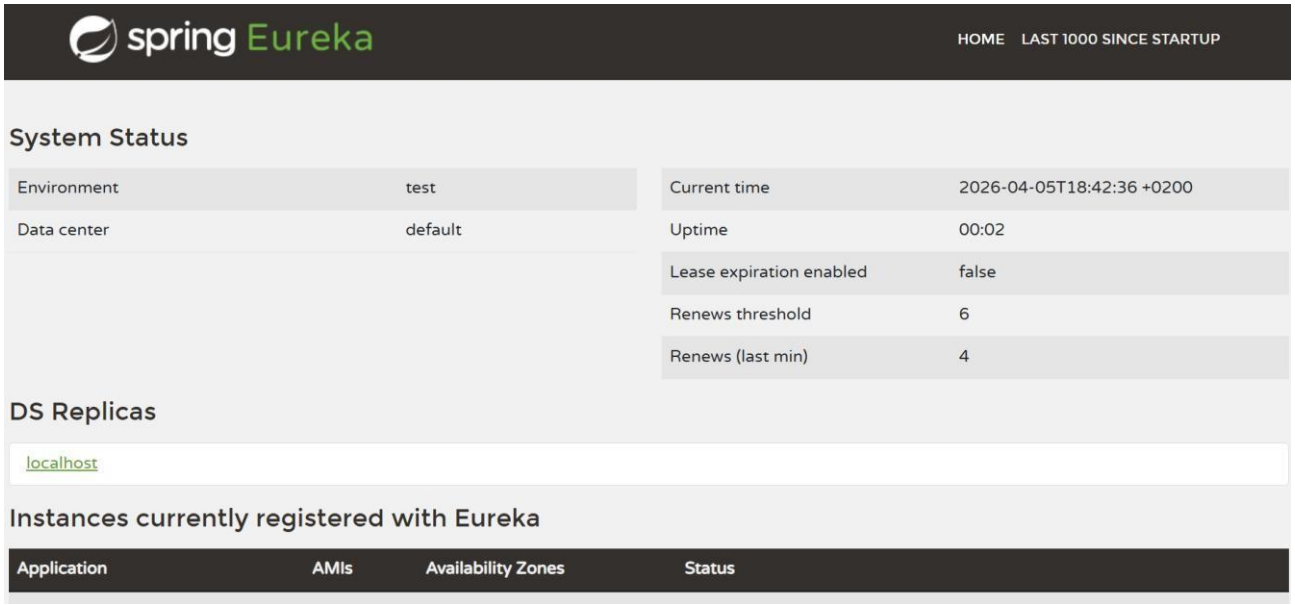
Étape 1 : Mise en place du Serveur de Découverte (Eureka)

1-Configuration

Cette étape consiste à configurer le serveur Eureka via le fichier **application.properties**. Le port du serveur est défini sur **8761**, et son nom d'application est **eureka-server**. Les options register-with-eureka et fetch-registry sont désactivées, car ce service joue uniquement le rôle de registre et ne s'enregistre pas lui-même ni ne récupère d'autres services. Enfin, l'URL de base d'Eureka est définie pour permettre aux autres microservices de s'y connecter. Cette configuration permet de mettre en place un service central de découverte des microservices dans l'architecture SOA.

2-Démarrage

Une fois la configuration terminée, l'étape finale consiste à compiler et à lancer l'instance du serveur de découverte pour rendre le registre disponible aux futurs microservices



Étape 2 : Créer le service Product

1-Configuration

On met en place le Product Service, un microservice chargé de gérer le catalogue des produits. Le projet est créé via Spring Initializr avec les dépendances nécessaires pour exposer des API REST, gérer la base de données et s'enregistrer auprès d'Eureka. La configuration définit un port spécifique (8081), un nom de service pour l'identification dans Eureka, ainsi que les paramètres de la base de données H2 en mémoire. Cette étape permet d'avoir un service autonome capable de stocker et gérer les produits tout en étant intégré à l'architecture SOA.

2-Modèle de données

On définit le modèle de données Product qui représente un produit dans l'application. Cette classe est annotée avec **@Entity** pour indiquer qu'elle correspond à une table dans la base de données, et contient les attributs essentiels comme l'identifiant, le nom, la description, le prix et le stock. Les annotations de Lombok simplifient la création des getters, setters et constructeurs. Ce modèle permet de structurer et stocker les informations des produits de manière organisée dans la base de données.

3-Repository

Dans cette partie, on crée le **repository** chargé d'accéder aux données des produits. L'interface **ProductRepository** étend **JpaRepository**, ce qui permet de bénéficier automatiquement des opérations CRUD (ajout, lecture, suppression, mise à jour) sans écrire de code supplémentaire. L'annotation **@Repository** indique que ce composant interagit avec la base de données. Cette couche facilite ainsi la gestion des données et la communication avec le modèle Product.

4-Service

Dans cette partie, on implémente la couche service qui contient la logique métier du microservice produit. La classe **ProductService** utilise le repository pour effectuer les opérations principales : récupérer tous les produits, chercher un produit par son ID, créer un nouveau produit et le supprimer. L'annotation **@Service** indique qu'il s'agit d'un composant métier, tandis que **@RequiredArgsConstructor** facilite l'injection du repository. Cette couche permet de séparer la logique métier de l'accès aux données pour une meilleure organisation du code.

5-Contrôleur REST

Dans cette partie, on crée le contrôleur REST qui expose les API du service produit. La classe **ProductController** permet de gérer les requêtes HTTP pour afficher, ajouter et supprimer des produits en s'appuyant sur le **ProductService**. Elle définit les endpoints principaux de l'API et sert d'interface entre le client et la logique métier.

6-Activation Eureka Client

Dans cette partie, on active le client Eureka en ajoutant l'annotation **@EnableDiscoveryClient** dans la classe principale. Cela permet au service Product de s'enregistrer automatiquement auprès du serveur Eureka et d'être découvert par les autres microservices. Cette configuration facilite la communication et l'interaction dans l'architecture SOA.

3.2 POST /api/products — Création via DTO

Après refactoring, la création d'un produit passe par le CreateProductRequest. Ici on crée "PC" à 12000,00 avec un stock de 3. La réponse est un ProductDTO (200 Created) — l'entité JPA n'est plus jamais exposée. Donc pas de stock a la sortie.

Étape 3: Créer le service Order

1-Configuration

Dans cette partie, on crée le service **Order** en suivant le même processus que le service Product, mais avec des dépendances supplémentaires comme OpenFeign pour permettre la communication entre microservices. La configuration définit le port d'exécution (8082), le nom du service pour Eureka, ainsi que les paramètres de la base de données H2 en mémoire. Cette étape permet de mettre en place un microservice autonome chargé de la gestion des commandes, tout en étant intégré à l'architecture globale.

2-Modèle de données

Dans cette partie, on définit le modèle de données **Order** qui représente une commande. La classe est annotée avec **@Entity** pour être persistée en base de données et contient les attributs nécessaires comme l'identifiant, l'ID du produit, le nom du produit, la quantité, le prix total, la date et le statut de la commande. Les annotations Lombok simplifient la gestion du code. Ce modèle permet de structurer et stocker les informations liées aux commandes.

3-Client Feign pour Product Service

Ici, on met en place un client Feign pour permettre au service **Order** de communiquer avec le service Product. L'interface **ProductClient** utilise **@FeignClient** pour appeler directement les API du service product-service via son nom enregistré dans Eureka. Elle permet de récupérer les informations d'un produit à partir de son identifiant, facilitant ainsi la communication entre microservices de manière simple et transparente.

4-DTO pour Product

Dans cette partie, on crée un **DTO (Data Transfer Object)** pour représenter les données du produit lors des échanges entre microservices. La classe **ProductDTO** permet de transférer uniquement les informations nécessaires sans exposer directement l'entité de la base de données. Les annotations Lombok simplifient la création des getters, setters et constructeurs. Ce DTO facilite ainsi la communication et le découplage entre les services.

5-Repository et Service

OrderRepository hérite de JpaRepository, ce qui permet de gérer facilement les opérations sur la base de données (ajout, suppression, recherche…) pour l'entité Order sans écrire de code SQL.

6-Contrôleur

Cette étape consiste à créer le contrôleur REST du service Order . OrderController expose les endpoints API pour interagir avec les commandes :

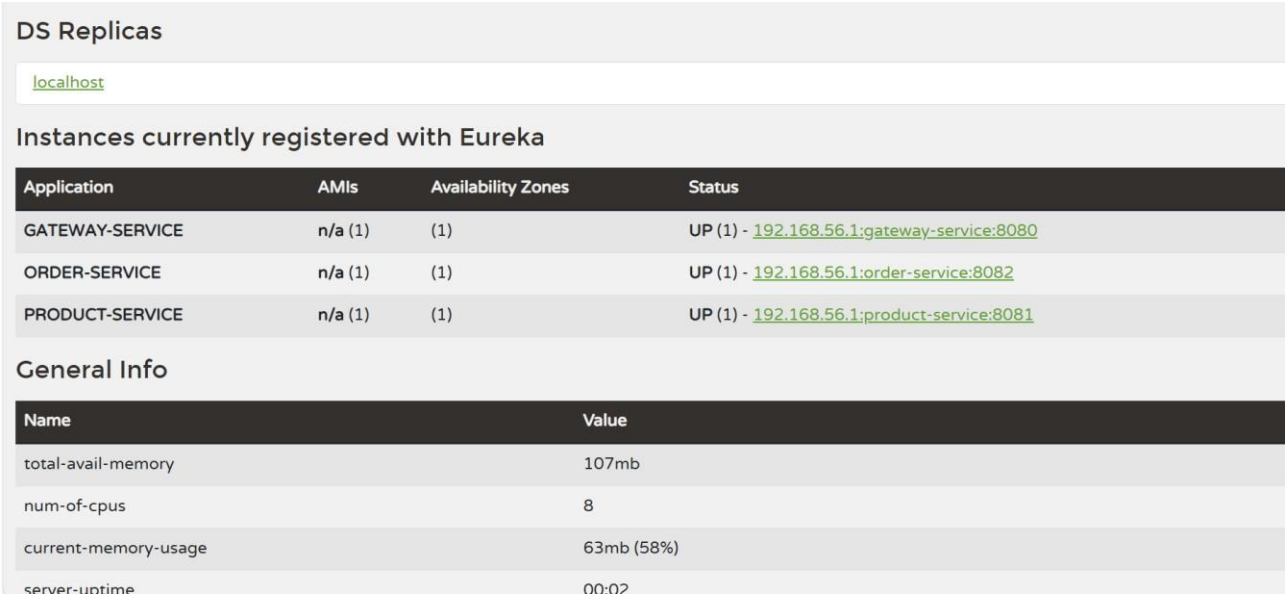
- GET /api/orders : récupère la liste de toutes les commandes.
- POST /api/orders : crée une nouvelle commande en appelant le service métier (OrderService).

7-Activation des clients Feign

L'activation de Feign et du Discovery Client marque la fin de la phase de développement du order-service. Le microservice est maintenant prêt à consommer les données du productservice de manière synchrone, illustrant le succès de la mise en réseau des composants de l'application e-commerce.

Étape 4 : Tester l'application

Les tests effectués avec Postman démontrent la robustesse de l'architecture. Le découplage est total : chaque service gère ses données, mais collabore de manière transparente grâce à OpenFeign. L'application e-commerce est désormais fonctionnelle, scalable et prête à recevoir de nouveaux modules.



DS Replicas

localhost

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
GATEWAY-SERVICE	n/a (1)	(1)	UP (1) - 192.168.56.1:gateway-service:8080
ORDER-SERVICE	n/a (1)	(1)	UP (1) - 192.168.56.1:order-service:8082
PRODUCT-SERVICE	n/a (1)	(1)	UP (1) - 192.168.56.1:product-service:8081

General Info

Name	Value
total-avail-memory	107mb
num-of-cpus	8
current-memory-usage	63mb (58%)
server-uptime	00:02

4. Tests Postman

Test 1 : Créer un produit

POST {{base_url}} /api/products Send

Docs Params Auth Headers (8) Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2     "name": "Laptop",
3     "description": "Pc portable",
4     "price": 4500.00,
5     "stock": 4
6 }
```

Body 200 OK 3.12 s 193 B

{} JSON Preview Visualize

```
1 {
2     "id": 1,
3     "name": "Laptop",
4     "description": "Pc portable",
5     "price": 4500.0,
6     "stock": 4
7 }
```

Test 2 : Lister les produits

GET {{base_url}} /api/products Send

Docs Params Auth Headers (6) Body Scripts Settings Cookies

Query Params

	Key	Value	Desc...	Bulk Edit
	Key	Value	Description	

Body 200 OK 288 ms 195 B

JSON Preview Visualize


```
[
    {
        "id": 1,
        "name": "Laptop",
        "description": "Pc portable",
        "price": 4500.0,
        "stock": 4
    }
]
```

Test 3 : Créer une commande

POST http://localhost:8081/ | GET http://localhost:8081/a | POST http://localhost:8082/ | GET http://localhost:8082/a

http://localhost:8082/api/orders

POST http://localhost:8082/api/orders?productId=1&quantity=2

Docs | Params | Authorization | Headers 7 | Body | Scripts | Tests | Settings

Query Params

Key	Value
productId	1
quantity	2

Body | Cookies | Headers 5 | Test Results

JSON | Preview | Visualize


```
1  {
2      "id": 1,
3      "productId": 1,
4      "productName": "Laptop",
5      "quantity": 2,
6      "totalPrice": 2400.0,
7      "orderDate": "2026-03-27T21:35:18.3193492",
8      "status": "PENDING"
9  }
```


Test 4 : Lister les commandes

POST http://localhost:8081/ | GET http://localhost:8081/a | POST http://localhost:8082/ | GET http://localhost:8082/a

http://localhost:8082/api/orders

GET http://localhost:8082/api/orders

Docs | Params | Authorization | Headers 6 | Body | Scripts | Tests | Settings

Body | Cookies | Headers 5 | Test Results

JSON | Preview | Visualize


```
1  [
2      {
3          "id": 1,
4          "productId": 1,
5          "productName": "Laptop",
6          "quantity": 2,
7          "totalPrice": 2400.0,
8          "orderDate": "2026-03-27T21:35:18.319349",
9          "status": "PENDING"
10     }
11 ]
```

Exercises Pratiques

Exercice 1 : Ajouter une fonctionnalité

Dans le Product Service (Le Fournisseur)

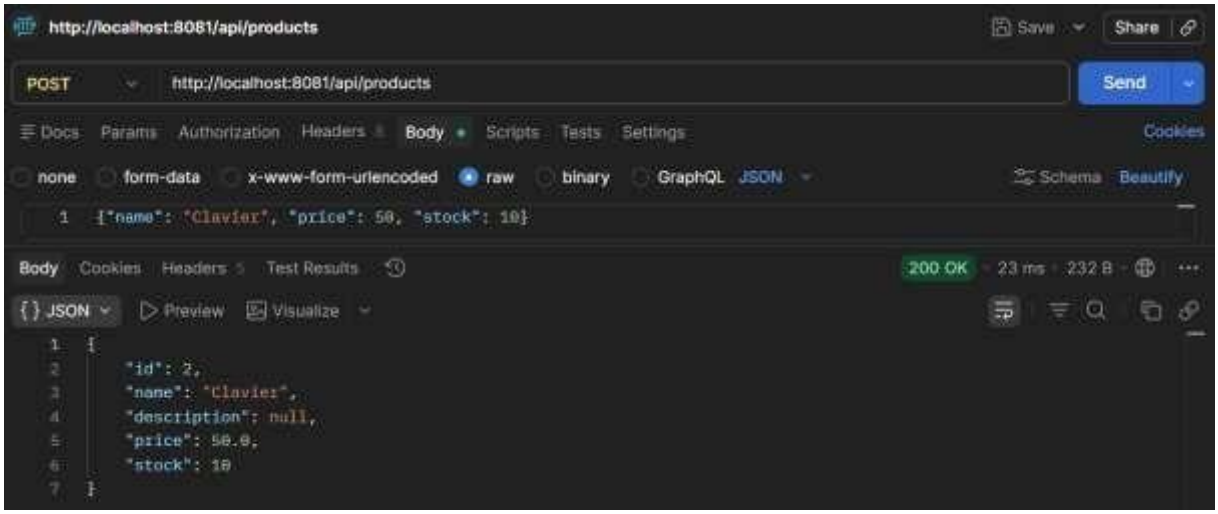
Fichier ProductService.java : Ajout une méthode pour modifier le stock.

Fichier ProductController.java : Création de l'endpoint PUT

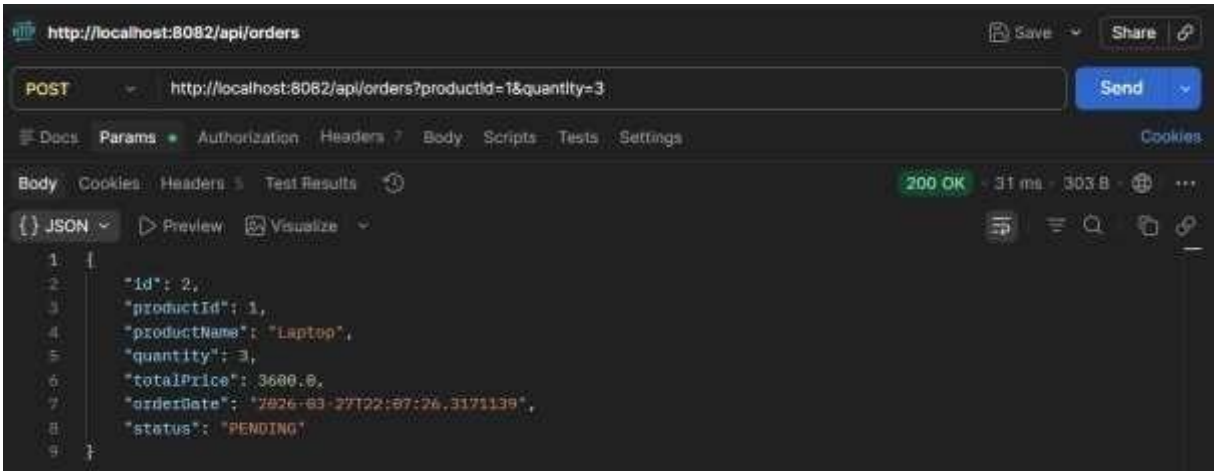
Dans le Order Service (Le Consommateur)
Fichier ProductClient.java (Feign) : Ajout de la signature de la méthode PUT.

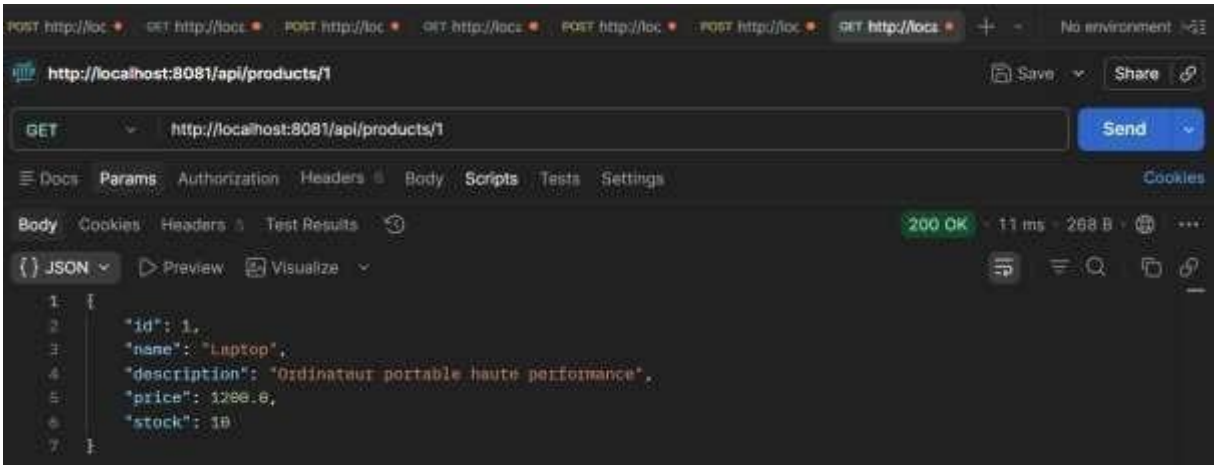
Fichier OrderService.java : Modification de la méthode de création de commande.

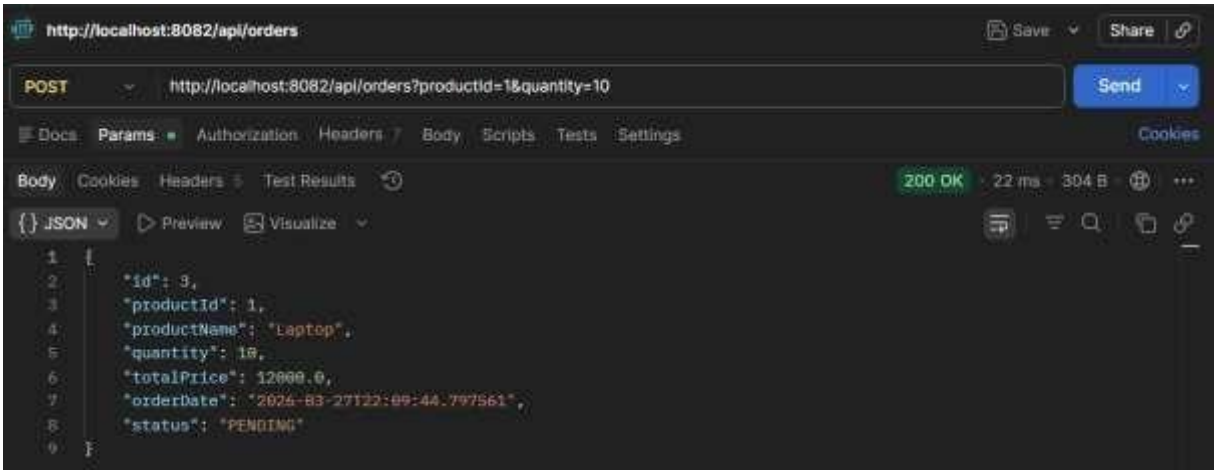
Teste 1-Initialiser un produit avec du stock



2-Test du scénario nominal (Succès)







Le succès de ces tests démontre le couplage fort entre la validation du stock et la création de la commande. L'utilisation d'OpenFeign permet ici de simuler une transaction distribuée simple : l'échec de la mise à jour du stock entraîne l'échec de l'ensemble du processus de commande. C'est une illustration concrète de la gestion de l'intégrité des

données dans une architecture microservices sans utiliser de gestionnaire de transactions complexes (comme SAGA).

Exercice 2 : Créer une API Gateway

1-Dépendances (pom.xml)

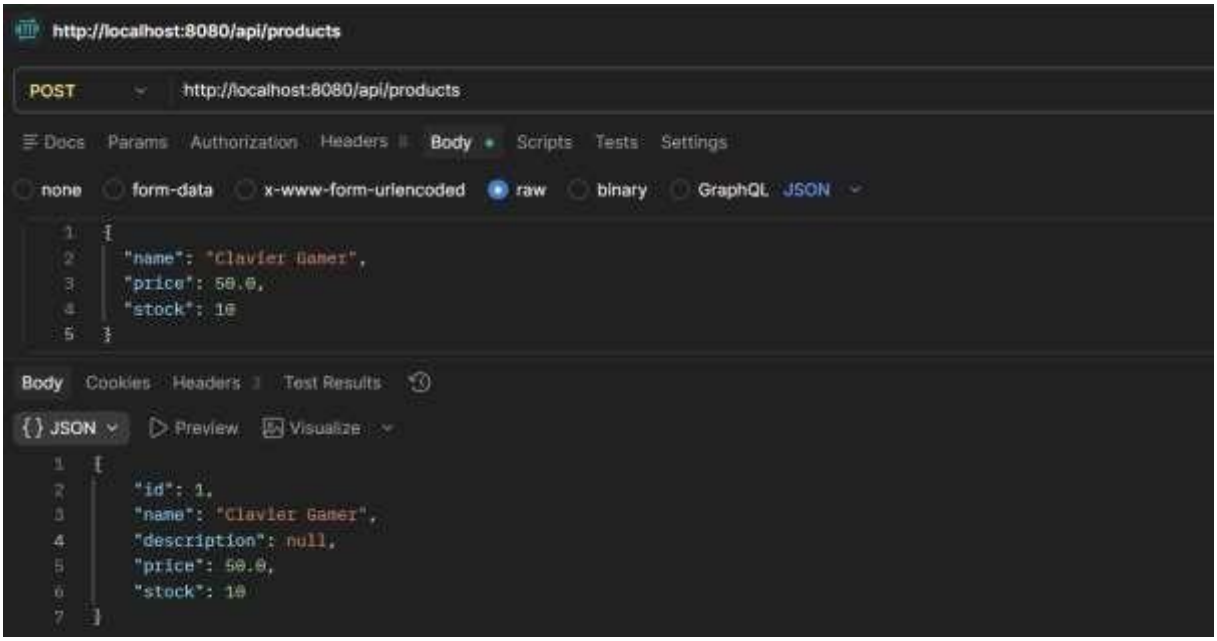
À l'intérieur de ce nouveau dossier gateway-service, crée un fichier nommé pom.xml

2-Configuration (application.properties)

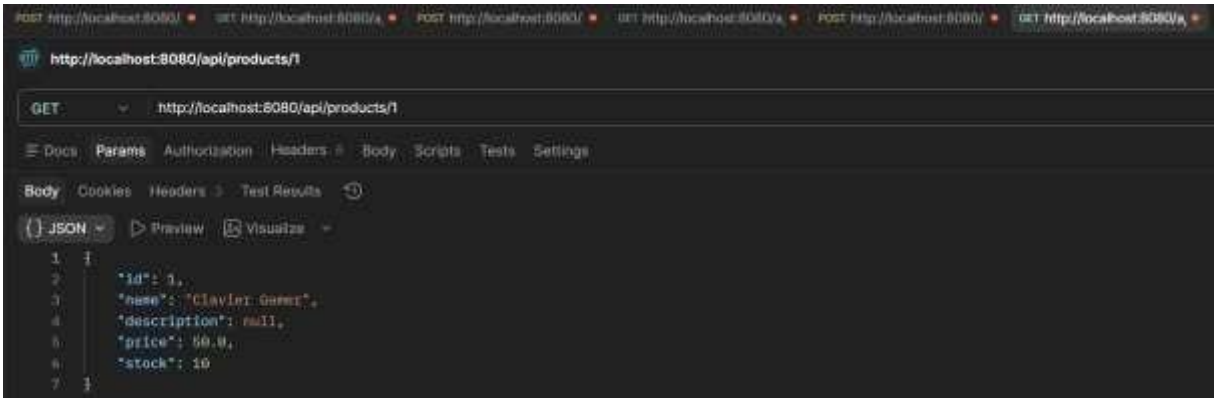
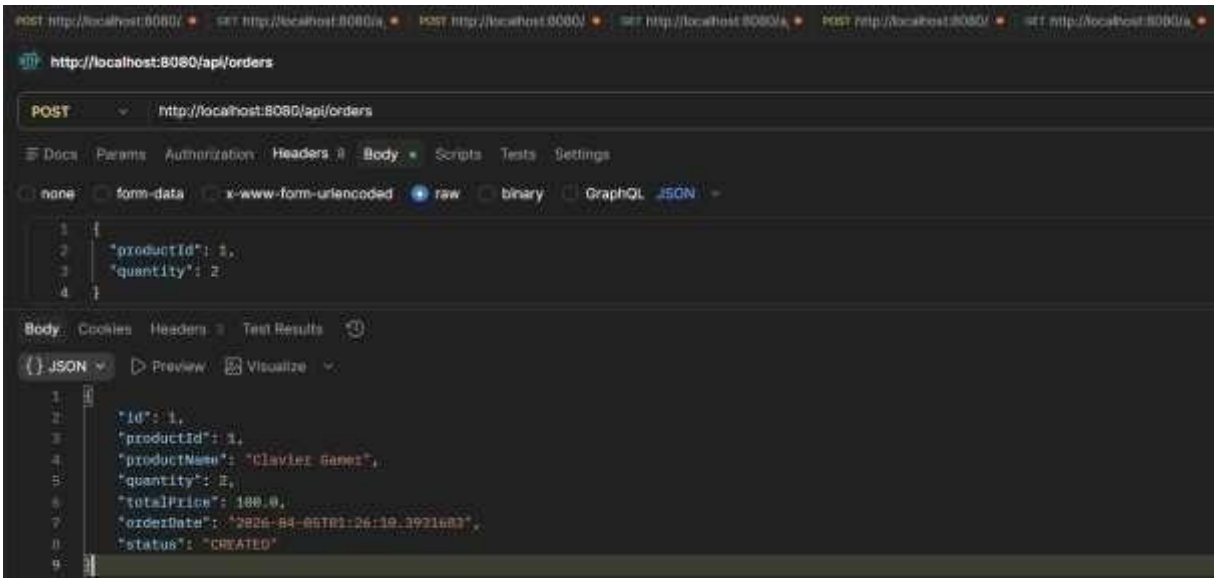
dans le dossier gateway-service/src/main/resources/ et crée (ou modifie) le fichier application.properties

Teste :

Créer le produit :



Vérifier que le produit existe :



Passer la commande :